

Semester Project Report

SAADE Tarek

Juin 2025

Introduction

In data science, particularly within biomedical research, we frequently encounter dynamic biological processes captured only as static, cross-sectional snapshots across different time points. Single-cell RNA sequencing (scRNA-seq) data exemplifies this challenge, providing rich information about cellular states at specific moments but destroying cells upon measurement, thus preventing direct observation of continuous trajectories. While methods based on optimal transport have been used to align these discrete snapshots, they often struggle to model the underlying continuous dynamics and the non-linear paths cells take during differentiation or other transitions.

This project aims at exploring and reproducing the results of **TrajectoryNet** [2], a method that addresses this challenge by establishing a principled link between Continuous Normalizing Flows (CNFs) and Dynamic Optimal Transport (DOT). TrajectoryNet leverages the power of CNFs to learn a continuous-time velocity field that transforms a simple base distribution into the complex distributions observed in scRNA-seq data over time. By incorporating regularizations inspired by optimal transport principles (like penalizing path energy) and biological priors (such as density preferences and external velocity information), TrajectoryNet guides the CNF to learn plausible and smooth cellular trajectories that approximate a dynamic optimal transport plan, even accounting for changes in cell population size (unbalanced transport).

The following sections will detail the core concepts of CNFs and Dynamical Optimal Transport as applied in this context, describe the architecture and implementation of the TrajectoryNet-based model, elaborate on the specific learning process including the various regularization strategies employed, and then try to apply the implemented model to various scenarios to explore its capabilities, evaluate its performance, and identify potential limitations in inferring continuous cellular dynamics from discrete scRNA-seq data.

1 Problem Formulation

The model’s foundation lies in two key concepts:

- **Dynamical Optimal Transport:** Building upon the concept of Optimal Transport as a mathematical framework for finding the most efficient way to move mass between distributions, Dynamical Optimal Transport, as explored by Benamou & Brenier (2000) [1], extends this to model the continuous evolution of distributions over time. This formulation links optimal transport distances with dynamics governed by partial differential equations (PDEs). For a fixed time interval $[t_0, t_1]$, it seeks a time-dependent density field $P(x, t)$ and velocity field $f(x, t)$ that satisfy the continuity equation:

$$\partial_t P + \nabla \cdot (Pf) = 0 \quad (1)$$

subject to the initial and final conditions $P(\cdot, t_0) = \mu$ and $P(\cdot, t_1) = \nu$. Under these constraints, the squared L^2 Wasserstein distance between the initial and final distributions (μ and ν) can be related to the infimum of the integrated squared L^2 norm of the velocity field:

$$W_2(\mu, \nu)^2 = \inf_{(P, f)} (t_1 - t_0) \int_{\mathbb{R}^d} \int_{t_0}^{t_1} P(x, t) |f(x, t)|^2 dt dx \quad (2)$$

The continuity equation ensures the conservation of mass throughout the process, asserting that no point acts as a source or sink. This formulation finds the velocity field with the minimum L^2 norm that transports the mass from the initial to the final distribution. While attractive for interpolating between known densities, traditional computational methods for solving this dynamic formulation often rely on space-time discretization, which scales poorly with the high dimensionality (d) characteristic of data like scRNA-seq, presenting a significant challenge. In the context of single-cell data, dynamical optimal transport provides a framework to infer the continuous evolution and correspondences between cell populations observed at different time points by finding the most efficient path of mass movement.

Observation: From this formulation, we can see potential limitations when applying it directly to complex biological systems. Analytically, the optimal velocity field in the L^2 Wasserstein case is related to the gradient of a convex potential function, which can often result in mapping points along straight lines in the high-dimensional space, especially between two distributions. We can anticipate that without modifications or additional constraints, a model solely minimizing the L^2 velocity norm might struggle to capture the often non-linear and intricate paths of cellular differentiation or other biological processes. For this formulation to better reflect real-world biological dynamics, further investigation is needed to identify

or learn potential fields that truly govern these complex transitions, or to incorporate information that acts as a corrector, such as observed velocity estimates.

- **Continuous Normalizing Flows (CNFs):** CNFs are a class of generative models that transform a simple base probability distribution (e.g., a standard Gaussian) into a complex target distribution via a learned, continuous-time transformation. This transformation is defined by an Ordinary Differential Equation (ODE):

$$\frac{dz(t)}{dt} = f_\theta(z(t), t)$$

where $z(t)$ represents a point in the data space at time t , and $f_\theta(z(t), t)$ is a vector field parameterized by a neural network with weights θ . The change of variables formula is crucial for density estimation with CNFs:

$$\log p(z(t_1)) = \log p(z(t_0)) - \int_{t_0}^{t_1} \text{Tr}(\nabla_z f_\theta(z(s), s)) ds$$

The integral of the divergence of the vector field, $\int_{t_0}^{t_1} \text{Tr}(\nabla_z f_\theta(z(s), s)) ds$, accounts for the change in log-probability density due to the transformation.

The model establishes a crucial link between these two concepts, adapting standard CNFs to approximate dynamic optimal transport. While standard CNFs primarily focus on maximizing the data likelihood, equivalent to minimizing the KL divergence between the learned distribution at the final time point and the target data distribution, they do not inherently minimize the transport cost or enforce a hard match at the target distribution.

TrajectoryNet modifies this by adding an energy regularization term to the standard CNF objective. This term is based on the integrated squared norm of the learned velocity field $\|f(x, t)\|^2$ along the trajectories. The theoretical basis for this lies in the connection between the L^2 Wasserstein distance (W_2^2) and the minimization of the integrated squared velocity under the continuity equation. By minimizing a combined objective that includes both the KL divergence at the final time point and a weighted penalty (λ_e) on the integrated squared velocity, TrajectoryNet approximates the dynamic optimal transport problem.

This is supported by theory suggesting that with a sufficiently large weight on the KL term (which is naturally handled by the CNF’s likelihood objective), minimizing this combined loss converges to the W_2^2 cost, effectively encouraging the flow to take low-cost paths characteristic of optimal transport.

$$W_2^2(\mu, \nu) = (t_1 - t_0) \inf_{(P, f)} \mathbb{E}_{x_0 \sim \mu} \left[\int_{t_0}^{t_1} \|f(x(t), t)\|^2 dt \right] + \lambda D_{KL}(P(\cdot, t_1) \| \nu); \quad \text{s.t.} \quad P(\cdot, t_0) = \mu \quad (3)$$

To further refine the learned paths and counteract the tendency for high energy penalties to cause "undershooting" of the target distribution, TrajectoryNet also incorporates a penalty on the integrated squared Frobenius norm of the Jacobian of the velocity field, $\|J_f(x, t)\|_F^2$, with weight λ_j . This Jacobian regularization discourages high local curvature, promoting smoother, straighter paths that are more consistent with simplified optimal transport solutions and can be intuitively thought of as penalizing acceleration.

Unlike traditional methods that might perform optimal transport sequentially between pairs of time points, potentially introducing discontinuities, TrajectoryNet learns a single, smooth velocity field defined over the entire time series. This allows the model to find a continuous and smooth minimal cost flow, better suited for modeling the inherently smooth dynamics of biological systems.

2 Model Architecture

The implemented code constructs a CNF model using the following components:

- **ODENet:** A neural network that parameterizes the vector field $f_\theta(z, t)$. The network takes the cell state z and time t as input and outputs the predicted velocity vector. It consists of three fully connected layers of 64 nodes with leaky ReLU activations.
- **ODEfunc:** A wrapper around the ODENet that handles the computation of the vector field output and its divergence. The divergence can be computed exactly or approximated using techniques like Hutchinson's estimator with Rademacher noise for higher-dimensional data.
- **CNF Layer:** This layer integrates the ODE defined by the ODEfunc over a specified time interval using an ODE solver (e.g., 'dopri5', 'adams'). It takes initial states and integration times as input and outputs the transformed states and the accumulated log-determinant of the Jacobian (`delta.logp`).
- **SequentialFlow (Optional):** Multiple CNF layers can be chained together (as specified by `args.num_blocks`), potentially with batch normalization layers (`args.batch_norm`) in between, to increase the model's capacity and expressiveness.
- **GrowthNet (Optional):** An optional pre-trained model (`growth_model`) to predict cell growth or death rates as a function of cell state and time. This component is essential for handling "unbalanced" optimal transport, where the total number of cells (or mass of the distribution) changes over time due to biological processes like proliferation and apoptosis.
- **Regularization Functions:** Various functions (`create_regularization_fns`) to compute regularization terms based on the learned vector field and its properties (e.g., L1/L2 norms, Jacobian Frobenius norm).

- **Spectral Normalization (Optional):** Can be applied to the weights of the ODEnet (`add_spectral_norm`) to control its Lipschitz constant, promoting training stability and generalization.

3 Learning Process

This section describes how the parameters of the TrajectoryNet model are learned from observed scRNA-seq data across multiple time points. The process is based on maximizing the likelihood of the data under the continuous normalizing flow defined by the model’s neural network, incorporating regularization terms to guide the learning towards solutions approximating dynamic optimal transport.

The learning process aims to determine the parameters θ of the neural network defining the vector field $f_\theta(x, t)$ by maximizing the likelihood of the observed scRNA-seq data distributions across all time points $t_1, \dots, t_k$. This is performed iteratively using an optimization procedure, such as the ADAM optimizer.

For each training step, samples are drawn from the empirical data distributions at different time points. To compute the loss, which includes maximizing the model’s likelihood of generating these observed samples (relative to a defined standard Gaussian base distribution at t_0) and minimizing various regularization terms, gradients are calculated via a **single backward integration** of the ODE using the adjoint method.

More specifically, a single training iteration involves the following steps:

1. **Backward Integration Setup:** The core computation proceeds by iterating backward through the defined integration time points $(t_k, t_{k-1}, \dots, t_0)$, starting from the latest observed time point t_k .
2. **Sampling and Concatenation:** At each time point t_i (from t_k down to t_1), a batch of cells x is sampled from the empirical data distribution at t_i . Optional training noise can be added to these samples. If t_i is not the very last time point (t_k), this batch x is concatenated with a batch z which consists of samples that have been transformed backward from the next future time point (t_{i+1}) down to the current time point t_i by the model’s flow. This ensures that the subsequent gradient calculation is based on points representing the flow from all later observed times, as well as the data directly observed at t_i .
3. **Backward Transformation Step:** The current batch of (potentially concatenated) samples at t_i is then transformed backward by one time step using the CNF model’s backward integration. This involves solving the ODE $dx/dt = f_\theta(x, t)$ backwards over the interval $[t_{i-1}, t_i]$. The result is a batch of samples z at t_{i-1} , and simultaneously, the term $\int_{t_{i-1}}^{t_i} \nabla_x \cdot f_\theta(x(t), t) dt$ (or $\delta \log p$) is accumulated, which represents the change in

log-likelihood over this interval. These transformed samples z and accumulated log-likelihood changes are stored:

$$z, \delta_logp = \text{model}(x, \text{zero}, \text{integration_times} = \text{integration_times})$$

4. **Base Distribution Likelihood:** After iterating backward through all time points, samples from the latest time point (t_k) have been transformed all the way back to the initial reference time, conceptually t_0 . The log-likelihood of these final backward-transformed samples is calculated under the predefined base distribution (e.g., the standard normal distribution at t_0).
5. **Accumulating Total Log-Likelihoods:** The stored $\delta \log p$ values (corresponding to the backward transformations over each interval) are used to reconstruct the total log-likelihood of the original data samples at each observed time point t_i relative to the base distribution at t_0 :

$$\text{logpx} = \text{logps} + \delta_logp$$

6. **Total Loss Computation:** The overall loss function for the current model parameters is computed. The primary component is typically the mean of the negative log-likelihoods across the relevant observed time points ($t_1, \dots, t_k$).

Regularization: Several regularization terms are added to this base loss (making the total loss L_T) to guide the learning process and incorporate biological priors:

- **Energy Regularization (L_{energy}):** Weighted by coefficients λ_e and λ_j , this term penalizes the integral of the squared norm of the vector field $\|f\|^2$ and the squared Frobenius norm of its Jacobian $\|J_f\|_F^2$ over time. The penalty on $\|f\|^2$ encourages shorter, less energetic paths, approximating the path length constraint in dynamic OT. The penalty on $\|J_f\|_F^2$ discourages high local curvature, analogous to penalizing acceleration:

$$L_{\text{energy}}(x) = \lambda_e \int_t \|f(\tilde{x}, t)\|^2 dt + \lambda_j \int_t \|J_f(\tilde{x})\|_F^2 dt$$

- **Density-Based Regularization (L_{density}):** (If used) Encourages transformed samples to remain close to the data manifold by penalizing points far from their k-nearest neighbors in the full dataset:

$$L_{\text{density}}(x, t_d) = \sum_k \max(0, \min\{\|x(t_d) - z\| : z \in X\} - h)$$

- **Velocity Regularization (L_{velocity}):** (If external velocity data is available) Penalizes the discrepancy (e.g., using cosine similarity) between the learned vector field f_θ and the known velocity vectors at

data points:

$$L_{\text{velocity}}(x, t, \frac{dx}{dt}) = \text{cosine-similarity}(f(x, t), \frac{dx}{dt})$$

- **Growth Rate Regularization (L_{growth}):** (If a growth model is used) Adjusts the log-likelihood calculation based on predicted growth rates to account for changes in mass over time:

$$\log M_{t_i}(x) = \log M_{t_{i-1}}(x) - \int_{t_{i-1}}^{t_i} \text{Tr} \left(\frac{\partial x(t)}{\partial f_{\theta}(x(t), t)} \right) dt - \log G(x_{t_{i-1}}, t_{i-1})$$

- **Other CNF Regularizations:** Additional standard CNF regularizations (e.g., L1/L2 penalties on parameters or norms of f or its Jacobians integrated over time) can also be included.
- **Total Loss Function:**
The total loss function is a sum of the negative log-likelihood and the weighted regularization terms:

$$L_T = - \sum_i \log P_{t_i}(x_{t_i}) + L_{\text{energy}} + L_{\text{density}} + L_{\text{velocity}} + L_{\text{growth}}$$

7. **Backpropagation and Optimization:** The gradient of the total loss L_T with respect to the model parameters θ is computed using backpropagation through the ODE solver (the adjoint sensitivity method). The optimizer then uses these gradients to update the model parameters θ , iteratively improving the model's fit to the data and reducing the overall loss.

Modifications to Standard CNF:

TrajectoryNet adapts standard Continuous Normalizing Flows (CNFs) to effectively model dynamics across multiple empirical time points, departing from the typical CNF framework which models transport between only two distributions. These adaptations are crucial for applying CNFs to time-series empirical data and for their use in approximating dynamic optimal transport.

The key modifications implemented in TrajectoryNet are:

1. **Modeling the Entire Time Evolution with a Single Smooth Flow:** Instead of training separate CNFs for sequential pairs of time points, a method that would be computationally costly and introduce artificial discontinuities, TrajectoryNet utilizes a single continuous vector field $f_{\theta}(x, t)$ defined by the neural network across the entire time range $[t_0, t_k]$. This design choice is motivated by the need to model underlying dynamics that are often smooth in time, particularly in biological systems, and provides a computationally more efficient representation of the continuous flow.
2. **Using a Defined Base Distribution:** Standard CNFs require knowing the probability density of the source distribution at t_0 to compute the

log-likelihood for training, based on the change of variables formula. However, when working with empirical data, the true density at t_0 is typically unknown. To address this, TrajectoryNet defines an artificial, known base distribution at t_0 , specifically the standard Gaussian $N(0, 1)$. The model is then trained to map this well-defined distribution to the observed data distributions at later time points. This allows the necessary log-likelihood terms for the loss calculation, involving the density at the origin $x_{t_0}^{(i)}$, to be computed.

3. Implementing a Specialized Backward Pass for Multi-timepoint

Data: The training process incorporates a unique strategy for computing gradients via the adjoint method when using data from discrete time points $(t_1, \dots, t_k)$. This involves integrating a batch backward from t_{i+1} to t_i and then concatenating it with a batch of empirical data samples observed at t_i . This combined batch is then used for the next backward step. While no theoretical justification is provided, the intuition behind this "concatenate-and-solve" strategy is to serve these purposes:

- **Anchoring and Drift Prevention:** By periodically including actual empirical data samples at each time point during the backward integration, the process effectively "re-anchors" the gradient computation to the true data manifold. This is crucial because backward integration, especially over many steps or with complex learned dynamics, can accumulate numerical and model errors, causing the represented density to drift away from the real data. This anchoring prevents runaway divergence and mitigates error compounding.
- **Parallel Efficiency:** Combining backward-mapped points (originating from later data) and empirical samples into a single batch allows the subsequent computations within the backward ODE solve to be processed in parallel. This offers significant computational speedup in gradient calculation compared to handling these different sets of points separately.

Computational Trade-off: While this method is computationally advantageous by reducing the total number of ODE solves compared to some alternatives, it involves a trade-off. Residual errors can still accumulate over a large number of steps (k) or with stiff vector fields despite the anchoring, highlighting the need for careful tuning of parameters like the number of time points used and solver tolerances to balance speed and accuracy.

4 Experiments

4.1 Experimental Design and Motivation

Due to computational constraints that precluded reproduction of biological experiments from the original paper, we focused exclusively on artificial arch-

shaped data (Figure 1). This synthetic system was strategically chosen because:

- Its low dimensionality (2D trajectories) enabled feasible computation on limited hardware.
- The non-linear arch structure provided a clear visual testbed for evaluating regularization efficacy.
- Minimal time points (t_0 : start, t_1 : hidden arch, t_2 : end) reduced complexity while preserving the core challenge of inferring intermediate dynamics.

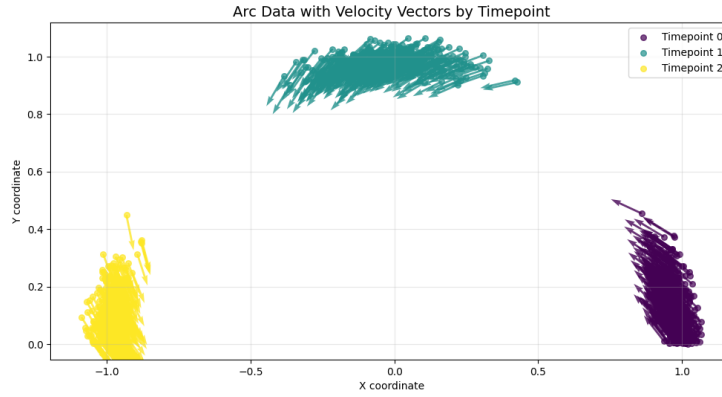


Figure 1: Arch Data

Key experimental strategy:

- **Critical hidden state:** The ground-truth arch distribution at t_1 was excluded during training, forcing the model to infer it solely from t_0 and t_2 .
- **Regularization focus:** We tested:
 - **Baseline:** No regularization ($\lambda_e = \lambda_j = \lambda_v = 0$)
 - **Velocity-guided:** $\lambda_v = 0.005$
- **Evaluation:** Quantified fidelity of the interpolated t_1 arch using:
 - Earth Mover’s Distance (EMD): Distributional similarity to ground truth
 - Mean Squared Error (MSE): Pointwise positional accuracy

This design directly probed whether endpoint data alone—augmented by regularization—could recover non-linear paths, or if explicit intermediate supervision was essential.

4.2 Implementation Details

Consistent with the original study, we used:

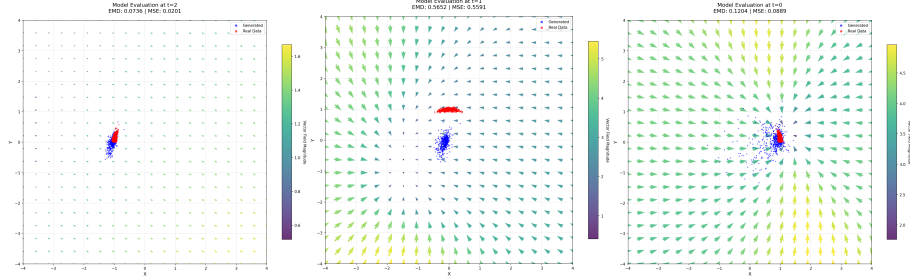
- **Base distribution:** Standard Gaussian $\mathcal{N}(0, I)$ centered at origin $[0, 0]$
- **Model:** 3-layer MLP (64 hidden units) with dopri5 ODE solver with both absolute and relative tolerances set to 1×10^{-5}
- **Parameters:** iterations = 900, batch size = 1,000, ADAM optimizer with learning rate 0.001, and weight decay 5×10^{-5}

Velocity data simulation: At t_0 and t_2 , we generated synthetic velocity vectors tangent to the true arch trajectory.

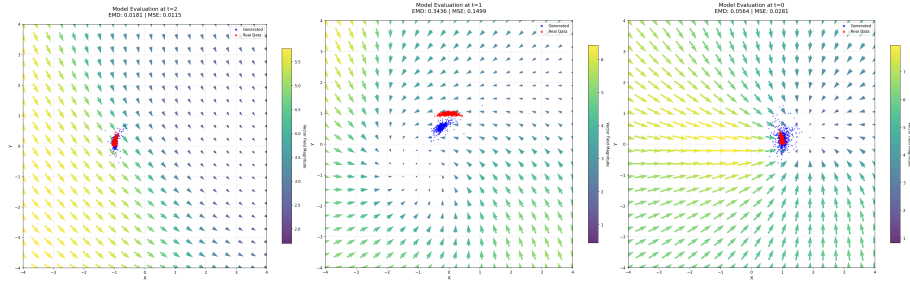
4.3 Key Findings

Qualitative results (Figure 2 and 3):

- **Without regularization:** Trajectories collapsed to straight lines, confirming theoretical expectations of minimal-energy OT paths.



- **With velocity regularization:** Recovered the arch structure with visual congruence to ground truth.



Quantitative results:

Condition	EMD (t_1)	MSE (t_1)
No regularization	0.5652	0.5591
Velocity regularization	0.3436	0.1499

Successes:

- Velocity regularization enabled accurate inference of hidden states.
- Validated the paper’s claim that external velocity cues counteract the ”straight-path bias” of OT.

Limitations and insights:

- **Velocity dependency:**
 - Accurate trajectories required velocity regularization.
 - This poses challenges for biological applications where RNA velocity is often noisy or unavailable.
- **Suboptimal base distribution:**
 - Initializing from $\mathcal{N}(0, I)$ forced the flow to first ”waste” energy moving points from $[0, 0]$ to the true t_0 distribution.
 - Attempts to initialize at the t_0 center of mass degraded performance due to training instability, suggesting the need for coordinated weight initialization schemes.
- **Jacobian regularization insufficiency:**
 - Without velocity cues, $\lambda_j \int \|J_f\|_F^2 dt$ reduced path oscillations but failed to induce curvature.

4.4 Conclusions

TrajectoryNet successfully inferred non-linear trajectories when guided by velocity regularization, confirming its theoretical capability to approximate dynamic OT. However, this strength is also a key limitation:

”The model’s performance hinges on access to accurate velocity data—a significant constraint in real-world biological applications where such information is sparse or unreliable.”

Future work should address **NON Euclidean** spaces or **NON Quadratic** cost function so the ”curves” arises naturally from the optimal transport solution without relying on external information (like velocity direction).

5 Contributions

My primary contribution involved:

- **Theoretical synthesis:** Conducted an in-depth analysis of the mathematical foundations, including:
 - The Benamou-Brenier formulation of Dynamic Optimal Transport
 - Continuous Normalizing Flows and their connection to ODE-based transformations
 - Regularization mechanisms (energy, Jacobian, velocity) and their biological interpretations
- **Reproducible code infrastructure:**
 - Established a dedicated Git repository structuring the original codebase
 - Implemented a structured interface for evaluation and visualization functions, allowing them to be passed as arguments for flexible experimentation
 - Added a function that plots vector fields alongside ground truth and sampled data points, including EMD and MSE metrics for quantitative comparison

```
def evaluate_visualize(device, args, model, timepoint):  
    "Generate samples, compute EMD and MSE, and visualize with vector fields"
```
 - Added a python notebook showing how to use the main code and the evaluation functions

References

- [1] Martin Burger, Ina Humpert, and Jan-Frederik Pietschmann. Dynamic optimal transport on networks, 2022.
- [2] Alexander Tong, Jessie Huang, Guy Wolf, David van Dijk, and Smita Krishnaswamy. Trajectorynet: A dynamic optimal transport network for modeling cellular dynamics, 2020.